Smart Blinds: Automatic Light Regulation System

Homework 3

Deivydas Tamulis

1 Problem and Goal

The goal of this project was to build a smart blinds prototype in the Tinkercad Arduino environment. The system measures ambient light using a photoresistor and controls a servo motor representing the position of window blinds. A potentiometer allows the user to choose the desired light level or directly adjust the blinds in manual mode.

A 16×2 LCD displays the measured light level and current settings. A push button switches between automatic and manual operation, while a red LED indicates that the servo has reached its maximum angle. The project demonstrates analog sensor input, servo control, an external interrupt, a hardware timer interrupt, direct GPIO register access, and EEPROM persistence.

2 System Design

The photoresistor is connected to analog input A0 and the potentiometer to A1. Both readings are converted from the Arduino's 0–1023 analog range into percentages from 0% to 100%. The system has two operating modes:

- **Automatic mode:** the potentiometer specifies the target light level. Every 200 ms, the controller compares the measured brightness with the target. If the difference exceeds 5 percentage points, it moves the servo by 5° toward the appropriate direction. Within the ±5 percentage-point tolerance, the angle remains unchanged.
- **Manual mode:** the potentiometer directly sets the servo position from 0° to 180°, independently of the light measurement.

The button on D2 toggles the operating mode. The LCD updates every 500 ms: its first row shows the measured light percentage, while the second shows either the manual angle or the automatic target and angle. The LED on D3 lights only when the commanded angle reaches 180°.

The operating mode and servo angle are retained in EEPROM after a period without changes.

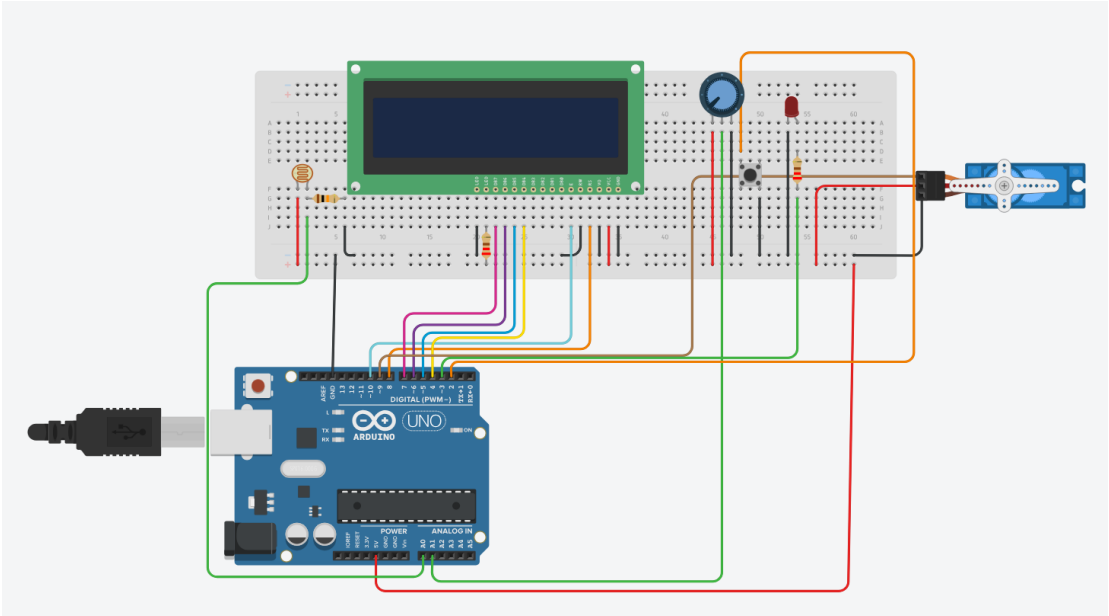


Figure 1: Complete smart blinds prototype assembled in Tinkercad.

3 Components and Connections

Component	Purpose	Arduino connection
Arduino Uno	Main controller	–
Photoresistor (LDR) + 10 kΩ resistor	Ambient light measurement / voltage divider	A0
Potentiometer	Desired light level / manual angle setting	A1
Micro servo	Adjusts the position of the blinds	Signal D9; 5 V and GND
16×2 LCD	Displays light level, mode and servo angle	RS D8, E D10, data D4–D7
Push button	Switches between AUTO and MANUAL	D2 to GND
Red LED + 220 Ω resistor	Indicates maximum servo angle	D3
LCD backlight resistor	Limits LCD backlight current	220 Ω
Breadboard and jumper wires	Electrical interconnection	Common 5 V and GND

Table 1: Main components and Arduino pin assignments.

The push button is configured as `INPUT_PULLUP`, so a press connects D2 to GND. The LED is controlled through the `DDRD` and `PORTD` registers rather than `digitalWrite()`. The servo and other components share a common ground.

4 Circuit Schematic

Figure 2 shows the electrical connections between the Arduino Uno, photoresistor voltage divider, potentiometer, servo motor, LCD, button and status LED. The LCD operates in 4-bit mode, using Arduino pins D4–D7 for data, D8 for register selection, and D10 for the enable signal. The analog sensors use A0 and A1, while the servo, button and LED use D9, D2 and D3, respectively.

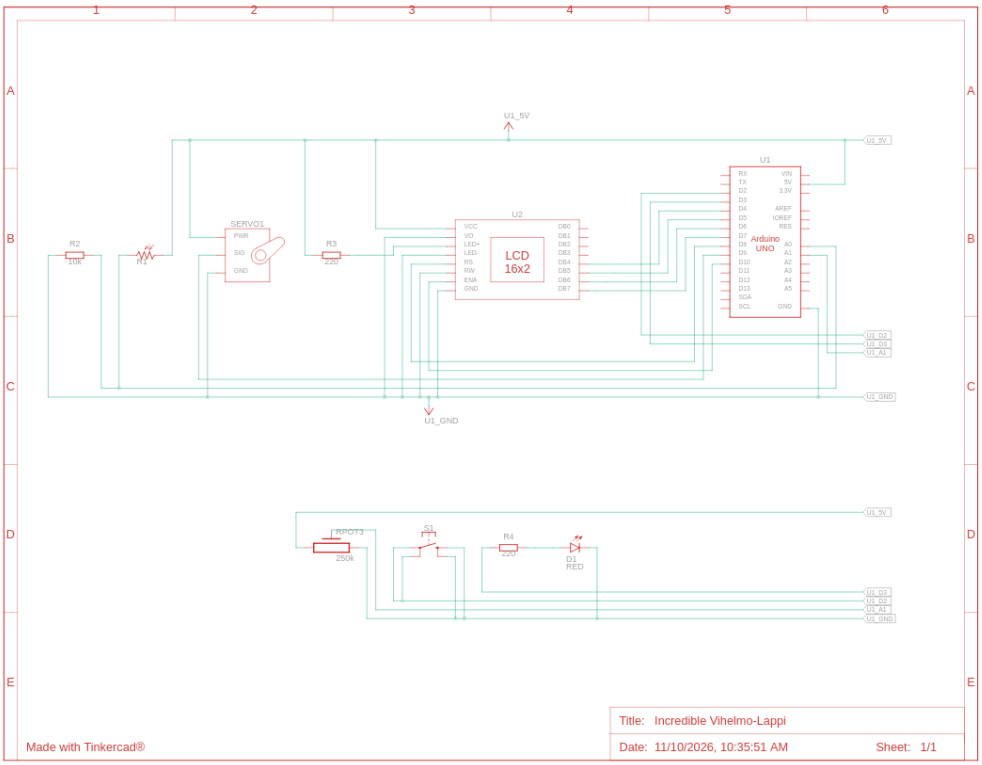


Figure 2: Tinkercad schematic of the smart blinds control circuit.

5 Testing and Results

The circuit and source code support the following functional checks in Tinkercad:

- In **AUTO**, change the photoresistor illumination and confirm that the servo angle changes in 5° steps when the measured light differs from the potentiometer target by more than 5 percentage points.
- Press the button and confirm that the system switches to **MANUAL**; rotating the potentiometer should then set the servo angle directly between 0° and 180°.
- Verify that the LCD reports the measured light level and the correct mode-specific values. At 180°, the red LED should turn on; at smaller angles it should turn off.
- Keep the settings unchanged for more than 3 seconds, restart the Arduino, and check that the stored operating mode and angle are restored.

The program uses the button external interrupt with 200 ms software debouncing. It stores the mode and servo angle in EEPROM after 3 seconds without changes and restores them at startup; without valid saved data, it starts in AUTO at 90°. A Timer2 interrupt schedules operations without `delay()`.

Timing: Timer2 operates in CTC mode with prescaler 64 and compare value `OCR2A = 249`. With a 16 MHz Arduino Uno clock, the interrupt interval is

$$T = \frac{64(249 + 1)}{16\,000\,000} = 0.001 \text{ s}.$$

The main loop reads the sensors and updates the servo every 200 ms, refreshes the LCD every 500 ms, and performs delayed EEPROM writes after 3000 ms of inactivity. Serial Monitor output at 9600 baud reports the light percentage, target percentage, servo angle and mode changes.

Limitations: The light percentage is derived from the analog sensor value and is not a calibrated measurement in lux. The LED indicates only the 180° position, not every possible open or closed state. EEPROM settings can be lost if power is removed before the delayed write takes place. The behaviors above are derived from the provided circuit and program; independent physical measurements were not supplied.

6 Possible Improvements

The system could be improved with calibrated light measurements, smoothing of photoresistor readings, and adjustable automatic-control thresholds. Additional buttons or an on-screen menu could make it possible to set the desired light level without reusing the same potentiometer. A dedicated power supply for the servo and an explicit calibration of the blinds' open and closed angles would also improve a physical implementation.